A. Object Design Patterns

- *Creational Patterns*
  - Abstract Factory
  - Builder
  - **Factory Method**
  - Prototype
  - Singleton
- *Structural Patterns*
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - **Façade**
  - Flyweight
  - Proxy
- *Behavioral Patterns*
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - **Observer**
  - State
  - **Strategy**
  - Template Method
  - Visitor

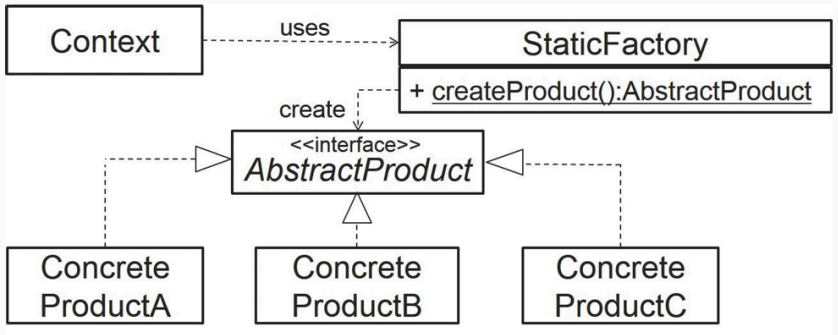
1.

- Defines an interface for creating different objects, without knowing beforehand what sort of objects it needs to create or how the object is created – lets subclasses decide which class to instantiate.

5

NANYANG TECHNOLOGICAL UNIVERSITY

Factory Pattern (a.k.a. Static Factory Pattern) – "Standard" Version (Template)

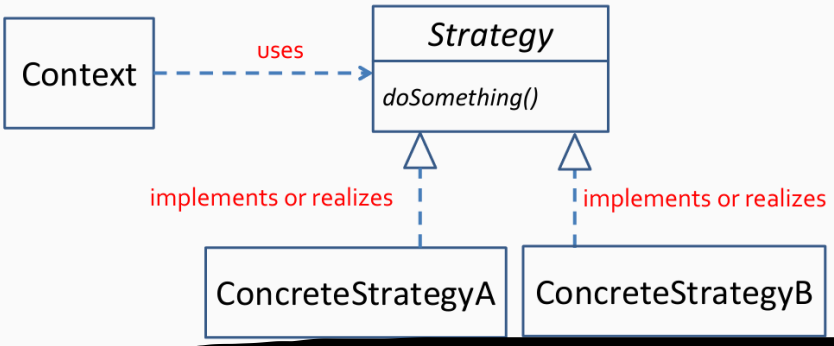


- **Context** uses **StaticFactory** to obtain an object that implements ***AbstractProduct*** interface.

2.

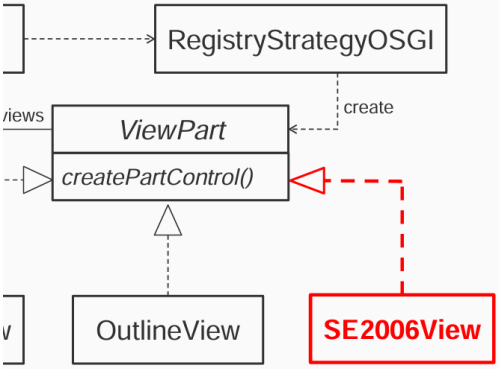
Strategy Pattern

- Design problem: A set of algorithms or objects should be interchangeable.
- Solution: Strategy Pattern



3.

esign + Dynamic Loading



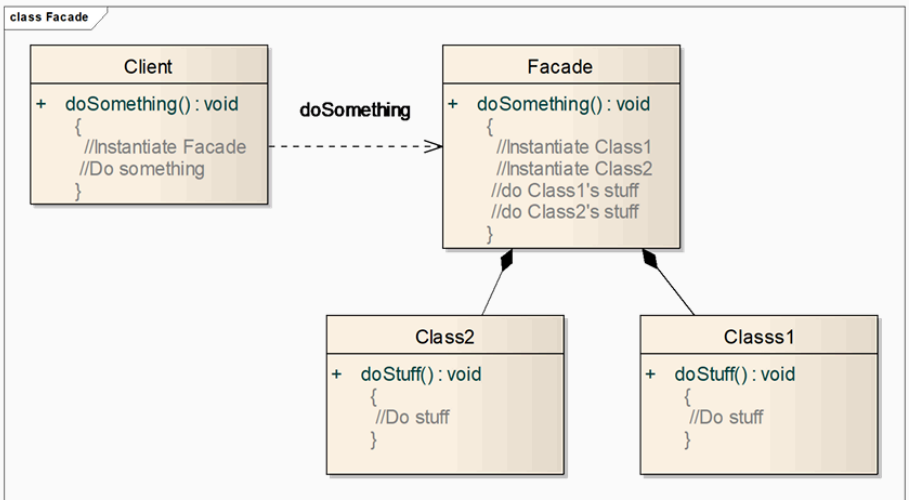
4.

Steps to implement Facade Design Pattern

Below are the simple steps to implement the Facade Design Pattern:

- **Step 1:** First, determine the complex subsystems or components that the client needs to interact with.
- **Step 2:** Build a facade class that serves as the middle layer between the client and the subsystems. This class should offer simplified methods that wrap the interactions with the subsystems.
- **Step 3:** Expose a clear, high-level interface in the facade class. This interface will include the methods that the client will use, hiding the internal complexity.
- **Step 4:** Inside the facade methods, delegate the requests to the appropriate subsystem classes to perform the actual operations.
- **Step 5:** The client now interacts only with the facade, which manages the underlying calls to the subsystems, simplifying the client's experience.

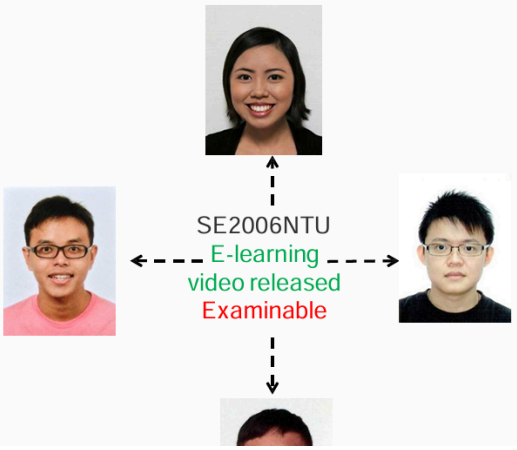
Façade – Class diagram



5.

When to Use the Observer Pattern

- When you need many objects (called observers) to receive an **update** when another object (called subject) **changes**



6.

Design Principles

High cohesion (like ProfileManager things)
Low coupling (loose coupling)
Open-Closed (open for extension but closed for modification)
Separation of concerns (seperate data access logic, business logic… into different layers.)